SIMATS ENGINEERING

CO4 - ASSESSMENT TOOL 2

Industry Problem Solving Task

COURSE NAME: Deep Learning **COURSE CODE: MLA0408**

COURSE OUTCOME COVERED

CO 1: Students will be able to understand and apply probabilistic graphical models including Bayesian Networks, Markov Random Fields, Hidden Markov Models, and Conditional Random Fields. They will learn how to represent complex probabilistic relationships among variables and perform inference and learning in graphical models. Students will be able to use these models for reasoning under uncertainty and solving real-world decision-making problems. (BL-6)

Assessment Tool Weightage: 40%

QUESTIONS: **Total Marks: 25**

Q1. Transfer Learning

A healthcare startup aims to develop an AI-based system for detecting skin cancer from dermoscopic images. Since only a limited number of labeled images are available, design a transfer learning strategy using a pre-trained CNN model. Justify your choice of model, identify which layers should be frozen or fine-tuned, and explain how your design improves classification performance.

Q2. DenseNet and PixelNet

A smart city project requires an automated road-scene analysis system that accurately identifies roads, pedestrians, vehicles, and traffic signs at the pixel level. Design a deep learning architecture using DenseNet and PixelNet to solve this problem. Explain how the proposed architecture improves feature reuse, segmentation accuracy, and computational efficiency.

Q3. Bidirectional RNN and Encoder–Decoder

A multinational customer service company wants to build a multilingual chatbot capable of translating customer queries while preserving contextual meaning. Design an Encoder–Decoder sequence-to-sequence model using Bidirectional RNNs. Illustrate the architecture and justify how bidirectional processing improves translation accuracy.

Q4. BPTT and LSTM

A financial analytics company is developing a model to forecast stock prices using historical market data. Create an LSTM-based prediction system and explain how Backpropagation Through Time (BPTT) is used to train the network. Justify why LSTM is more suitable than a traditional RNN for this application.

Q5. Integrated Deep Learning Solution

A media streaming company plans to automatically generate captions for uploaded videos to improve accessibility. Design an integrated deep learning solution that combines Transfer Learning for visual feature extraction with an LSTM-based Encoder–Decoder architecture for caption generation. Explain the role of each component and justify how the complete system produces accurate captions.

Code of Conduct:

I **G. Naveen Kumar Reddy (192525288)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis

PART B: SOLUTIONS TO INDUSTRY PROBLEM SOLVING TASK

Each response below addresses the assessment rubric criteria: Understanding of the Industry Problem, Application of Engineering Concepts, Solution Design, Use of Tools, and Analysis.

Q1. Transfer Learning for Skin Cancer Detection

1. Understanding of the Industry Problem

A healthcare startup needs to classify dermoscopic images (e.g., melanoma vs. benign nevus vs. other lesion classes) but has only a small labelled dataset. Training a deep CNN from scratch on such limited data would overfit badly and fail to generalize to unseen patients. The constraint is therefore data scarcity combined with the need for high diagnostic reliability, since false negatives in cancer detection carry severe clinical risk.

2. Application of Engineering Concepts – Choice of Pre-trained Model

EfficientNet-B3 (or ResNet-50 as an alternative) pre-trained on ImageNet is recommended. ImageNet-pretrained networks already encode generic low- and mid-level visual features (edges, textures, colour gradients, shape boundaries) that transfer well to dermoscopic images, which share similar low-level statistics with natural images despite the domain gap. EfficientNet is preferred because its compound scaling (depth, width, resolution) gives a strong accuracy-to-parameter ratio, which matters when the fine-tuning dataset is small and overfitting risk is high.

3. Solution Design – Freezing and Fine-tuning Strategy

- Stage 1 (Feature extraction): Freeze all convolutional/backbone layers. Replace the final classification head with a new Global Average Pooling layer + Dense layer (256 units, ReLU) + Dropout (0.5) + Softmax output matching the number of lesion classes. Train only this new head for several epochs so it adapts to skin-lesion features without disturbing the pre-trained weights.
- Stage 2 (Fine-tuning): Unfreeze the last 20–30% of the convolutional blocks (the higher-level, task-specific layers) while keeping the early layers (which capture generic edges/textures) frozen. Fine-tune with a very small learning rate (e.g., 1e-5) to gently adapt high-level filters to lesion-specific patterns (irregular borders, colour variegation) without catastrophic forgetting.
- Regularization: Use aggressive data augmentation (rotation, flipping, zoom, colour jitter, elastic deformation) to synthetically expand the small dataset and reduce overfitting.
- Class imbalance handling: Apply class-weighted loss or focal loss, since malignant cases are typically under-represented compared to benign cases.

4. Use of Tools

Implementation uses TensorFlow/Keras or PyTorch with torchvision.models pre-trained weights, Albumentations for augmentation, and Grad-CAM for visual explainability so clinicians can verify that the model is focusing on the lesion region rather than artifacts (e.g., rulers, hair, skin markers) — an important trust requirement in healthcare deployment.

5. Analysis – Why This Improves Performance

Freezing early layers preserves generic feature detectors learned from millions of ImageNet images, which would be impossible to relearn from a small medical dataset. Fine-tuning only the deeper layers lets the network specialize to lesion-specific texture and colour cues while avoiding overfitting. Evaluation should use sensitivity, specificity, AUC-ROC, and F1-score (not just accuracy) because of class imbalance and the clinical cost of false negatives; k-fold cross-validation and a held-out patient-level test set confirm generalization.

Q2. DenseNet and PixelNet for Road-Scene Segmentation

1. Understanding of the Industry Problem

A smart-city road-scene system must perform dense, pixel-level semantic segmentation to simultaneously identify roads, pedestrians, vehicles, and traffic signs in real time from camera feeds. The challenge is achieving high segmentation accuracy for small/thin classes (e.g., traffic signs, pedestrians at a distance) while keeping computation low enough for edge deployment (traffic cameras, onboard units).

2. Application of Engineering Concepts

DenseNet is used as the feature-extraction backbone, and PixelNet-style dense prediction is used for the pixel-wise classification head, combined in an encoder–decoder segmentation architecture (similar in spirit to FC-DenseNet / Tiramisu).

3. Solution Design

- Encoder (DenseNet backbone): Each layer receives feature maps from all preceding layers via dense connections (concatenation, not summation). This maximizes feature reuse — low-level edge/colour information from early layers stays directly accessible to the deepest layers, which is critical for recovering fine boundaries of thin objects like traffic-sign poles or pedestrian silhouettes.
- Transition-down blocks: 1x1 convolution + pooling between dense blocks control feature-map growth and reduce spatial resolution progressively, keeping parameter count and computation manageable despite the dense connectivity.
- Decoder (PixelNet-style dense prediction head): Up-samples encoder features back to full resolution using transposed convolutions / transition-up blocks, with skip connections from corresponding encoder dense blocks (analogous to U-Net) to recover spatial detail lost during downsampling. A hypercolumn-style PixelNet head concatenates multi-scale features per pixel before the final per-pixel softmax classification into {road, pedestrian, vehicle, traffic sign, background}.
- Output: A dense per-pixel class map overlaid on the original frame, plus bounding regions extracted from connected components for downstream tracking.

4. Use of Tools

Built in PyTorch using torchvision DenseNet-121 pre-trained weights as the encoder, trained/fine-tuned on a real road-scene dataset such as Cityscapes or BDD100K, with mixed-precision training for efficiency and TensorRT/ONNX export for real-time inference on embedded traffic hardware.

5. Analysis

Dense connectivity improves feature reuse and gradient flow, reducing the number of parameters needed compared to a plain deep CNN of similar depth (each layer only needs to learn a small set of new feature maps, growth-rate k, rather than relearning full feature stacks) — this directly improves computational efficiency. The multi-scale skip fusion in the PixelNet-style head improves segmentation accuracy, especially mean Intersection-over-Union (mIoU) on small classes like traffic signs. Performance should be reported per-class IoU, mean IoU, and inference frames-per-second to validate the accuracy–efficiency trade-off required for a real-time smart-city deployment.

Q3. Bidirectional RNN Encoder–Decoder for Multilingual Chatbot Translation

1. Understanding of the Industry Problem

The customer-service chatbot must translate a customer query from a source language into the agent/target language while preserving contextual and semantic meaning — literal word-by-word translation is not acceptable because customer intent, tone, and idioms must be retained for accurate support resolution.

2. Application of Engineering Concepts

A sequence-to-sequence (Seq2Seq) Encoder–Decoder architecture is used, where the encoder is a Bidirectional RNN (Bi-LSTM/Bi-GRU) and the decoder is a unidirectional RNN with an attention mechanism.

3. Solution Design

- Input embedding: Customer query tokens are converted to dense embeddings (e.g., via subword tokenization such as BPE, to handle multiple languages and rare words).
- Bidirectional Encoder: A Bi-LSTM processes the sentence in both forward and backward directions. At each time step t, the forward LSTM reads tokens 1..t while the backward LSTM reads tokens n..t; their hidden states are concatenated to form a context-aware representation of each word that captures both preceding and following context — essential because meaning in language often depends on words that come later in the sentence (e.g., negation, relative clauses).
- Attention layer: Instead of compressing the whole sentence into a single fixed vector, an attention mechanism (Bahdanau or Luong style) lets the decoder, at each output step, attend to the most relevant encoder hidden states, improving translation of long or complex queries.
- Decoder: A unidirectional LSTM generates the target-language sentence one token at a time, conditioned on the attention context vector and previously generated tokens (teacher forcing during training).
- Output layer: Softmax over the target vocabulary produces the next translated token; beam search is used at inference for higher-quality decoding.

4. Use of Tools

Implemented using PyTorch/TensorFlow Seq2Seq APIs (or fine-tuning a pre-trained multilingual transformer such as mBART/NLLB for production), with SentencePiece/BPE tokenization and BLEU/METEOR evaluation tools.

5. Analysis

Bidirectional processing improves translation accuracy because each source word's representation incorporates full-sentence context rather than only the words seen so far, which resolves ambiguities (e.g., pronoun references, word sense) that a unidirectional encoder would miss. Combined with attention, this reduces the information bottleneck of a single fixed-length context vector, particularly improving performance on longer customer queries. Accuracy should be validated with BLEU score, human evaluation of contextual fidelity, and latency measurement to confirm real-time chatbot responsiveness.

Q4. LSTM and BPTT for Stock Price Forecasting

1. Understanding of the Industry Problem

Stock price forecasting requires modelling long, noisy, sequential time-series data (price, volume, technical indicators) where both short-term fluctuations and longer-term trends/dependencies influence the next value. A financial analytics company needs a model that captures temporal dependencies over many time steps without losing earlier signal.

2. Application of Engineering Concepts

An LSTM-based sequence model is designed to predict the next-step (or multi-step) closing price, trained using Backpropagation Through Time (BPTT).

3. Solution Design

- Input: A sliding window of the past N days' features (open, high, low, close, volume, moving averages, RSI) normalized via min-max or z-score scaling.
- Architecture: Two stacked LSTM layers (e.g., 128 then 64 units) with dropout between them, followed by a Dense layer that outputs the predicted next-day price (or a probability distribution over up/down movement for classification variants).
- Training via BPTT: The network is unrolled across all T time steps of the input window into an equivalent deep feed-forward computational graph. The loss (e.g., MSE) at the final (or each) time step is backpropagated through this unrolled graph, and gradients with respect to the shared weight matrices are accumulated across all time steps before a single weight update — this is BPTT. Truncated BPTT (unrolling only a fixed window, e.g., 30–60 days) is used in practice to bound memory and computation.
- Gradient stability: LSTM's gating mechanism (input, forget, output gates) combined with its additive cell-state update keeps gradients from vanishing across long unrolled sequences during BPTT, unlike a vanilla RNN.

4. Use of Tools

Implemented in PyTorch/Keras with real market data pulled via APIs (e.g., Yahoo Finance/Alpha Vantage), pandas for feature engineering, and walk-forward (rolling-window) validation appropriate for time-series rather than random train/test splits.

5. Analysis – Why LSTM over a Traditional RNN

A vanilla RNN suffers from vanishing/exploding gradients during BPTT because the same weight matrix is repeatedly multiplied through every time step, so gradients shrink (or blow up) exponentially over long sequences and the network cannot learn dependencies spanning more than roughly 10–20 steps. LSTM solves this with its forget/input/output gates and a separate cell state that is updated additively rather than multiplicatively at every step, allowing gradients to flow largely unchanged across long horizons. This makes LSTM far better suited to stock data, where relevant patterns (e.g., a trend reversal signalled weeks earlier) can span long time windows. Model quality should be validated using RMSE/MAE against a naive baseline (e.g., previous-day price) and directional accuracy, since raw price RMSE alone can be misleading in finance.

Q5. Integrated Deep Learning Solution: Automatic Video Captioning

1. Understanding of the Industry Problem

A media streaming company needs to automatically generate natural-language captions for uploaded videos to improve accessibility (e.g., for hearing-impaired viewers) at scale, without manual transcription. This requires understanding visual content across frames and producing grammatically correct, contextually accurate sentences describing the video — a joint vision-and-language problem.

2. Application of Engineering Concepts

The solution integrates Transfer Learning (for visual feature extraction) with an LSTM-based Encoder–Decoder (for language generation), forming a video-captioning pipeline.

3. Solution Design

- Component A – Visual Feature Extraction (Transfer Learning): Sample frames from each video at fixed intervals and pass each through a pre-trained CNN (e.g., InceptionV3 or ResNet-50, pre-trained on ImageNet) with the classification head removed, using it purely as a fixed or lightly fine-tuned feature extractor. This produces a sequence of high-level visual feature vectors, one per sampled frame, without needing to train a vision backbone from scratch.
- Temporal aggregation: The per-frame CNN features are fed as a sequence into a Bi-LSTM (or a temporal attention pooling layer) to build a single video-level context representation that captures motion and scene evolution across time, not just isolated frames.
- Component B – Caption Generation (LSTM Encoder–Decoder): The aggregated visual context vector initializes an LSTM decoder that generates the caption word-by-word, with an attention mechanism over the frame-level features so that each generated word can attend to the most relevant frames (similar to 'Show, Attend and Tell').
- Training: The CNN encoder (transfer-learned) and LSTM decoder are trained jointly (or the decoder trained first with a frozen encoder, then fine-tuned end-to-end at a low learning rate), using teacher forcing and cross-entropy loss against ground-truth captions from a dataset such as MSR-VTT or MSVD.
- Inference: Beam search decoding produces the final caption text, which is then time-aligned and formatted as subtitle/caption files (e.g., SRT/VTT) for the video player.

4. Use of Tools

Implemented in PyTorch/TensorFlow using pre-trained torchvision/Keras Applications CNNs, an attention-based LSTM decoder module, and evaluated with standard captioning tools/metrics; deployed as a batch inference pipeline that processes uploaded videos asynchronously and writes caption files to the streaming platform's storage.

5. Analysis – Role of Each Component and Why the Integration Works

Transfer learning solves the problem of learning strong visual representations without needing millions of labelled video frames — the pre-trained CNN already 'understands' objects, scenes, and actions from ImageNet and transfers that knowledge efficiently. The LSTM Encoder–Decoder solves the sequence-generation problem, converting a fixed or attended visual context into a fluent, grammatically correct sentence, learning language structure from the captioning dataset rather than from the (much smaller) video corpus. Together, this division of labour lets each component be trained efficiently with the data actually available for it. Caption quality is evaluated using BLEU, METEOR, CIDEr, and ROUGE-L scores against reference captions, plus qualitative human review, to confirm both visual grounding and linguistic accuracy before deployment.